We are extending the existing private repository `apws/260906-7xtgv`.

This task is about the **native 7xtgv display model**, NOT redesigning the currently working VT100/RunCPM terminal. Preserve the working VT100/RunCPM code unless a tiny shared abstraction is clearly beneficial. Do not perform speculative optimization loops. If an architectural choice not covered below materially affects the design, stop and ask the human before choosing.

Goal

Define and implement a clean reference version of the native 7xtgv cell/compositor architecture, concentrating first on **SEMI mode**, plus a desktop-side image converter that can demonstrate a photograph such as the classic colorful parrot test image.

The architecture is intentionally intended to remain simple enough to translate later into HDL.

Native cell memory

Each visible cell has four bytes:

- `CHAR_A` — 8 bits
- `CHAR_B` — 8 bits
- `ATTR` — 8 bits
- `EXTA` — 8 bits

For 80×24:

- 1920 cells
- 7680 bytes total cell RAM

Do not merge CHAR_A and CHAR_B conceptually. They are two independently meaningful glyph/mask planes.

There is also conceptually a separate **hard desktop/background pixel source** beneath the cell compositor. It may eventually be a full-resolution RGB image/framebuffer or generated background. The compositor should accept a background pixel; storage of that desktop is not part of the core cell specification.

ATTR format

General:

```
bits 7..6 = MM mode
```

Proposed native modes:

```
00 = TEXT
01 = WIND
10 = INST
11 = SEMI
```

For this task, fully implement/specify SEMI. Do not prematurely design all WIND/INST semantics.

EXTA in modes 00/01/10

For TEXT/WIND/INST:

```
EXTA = PPPPIIII
```

where:

- `PPPP` = paper/background brightness, 0..15
- `IIII` = ink/foreground brightness, 0..15

This brightness is separate from physical TFT LED-backlight brightness.

Intended uses include:

- dim inactive windows
- focus indication
- fades
- animated blinking/fading
- context switching
- presentation effects

EXTA is expected to be relatively stable and not rewritten constantly.

Brightness behavior should be specified independently from implementation. An HDL implementation should be able to use lookup tables rather than RGB multipliers.

Do not implement complicated alpha blending yet. Leave a clean path for later optional fade-to-desktop behavior.

SEMI geometry

SEMI uses a logical **2 columns × 5 rows** of semipixels per 6×10 physical character cell.

Therefore each logical SEMI pixel expands to exactly:

```
3 physical pixels × 2 physical pixels
```

An 80×24 screen therefore becomes:

```
160 × 120 logical SEMI pixels
```

This regular geometry is intentional.

Each binary SEMI plane needs 10 bits:

```
bit 0  bit 1
bit 2  bit 3
bit 4  bit 5
bit 6  bit 7
bit 8  bit 9
```

No 1024-glyph SEMI ROM is required. The 10-bit number IS the bitmap.

SEMI ATTR/EXTA encoding

For MM=11:

```
ATTR = 11CCAABB
```

Interpret:

- `AA` = high two bits of CHAR_A's 10-bit SEMI mask
- `BB` = high two bits of CHAR_B's 10-bit SEMI mask
- `CC` = high two bits of the SEMI palette index
- `EXTA` = low eight bits of the SEMI palette index

Thus:

```
maskA10 = (AA << 8) | CHAR_A
maskB10 = (BB << 8) | CHAR_B

paletteIndex10 = (CC << 8) | EXTA
```

All three therefore span 0..1023.

Keep AA and BB adjacent in the low nibble as `AABB`.

SEMI compositor policy

Support two conceptually separate SEMI interpretations. Do NOT consume more cell attribute bits to select between them. Treat this as a display/compositor configuration or register.

1. SEMI_OVERLAY — native/default philosophy

CHAR_A and CHAR_B remain independent 1-bpp overlays over the hard desktop.

For each logical semipixel:

```
A=0 B=0 -> expose underlying desktop/background
A=0 B=1 -> B plane color
A=1 B=0 -> A plane color
A=1 B=1 -> A plane color (A is above B)
```

Each of the 1024 SEMI palette entries therefore supplies at least:

```
colorA
colorB
```

This preserves the original 7xtgv idea of binary-mask compositing over a hard background.

2. SEMI_INDEXED4 — optional full-screen graphics/demo mode

Use A and B as a 2-bit color index:

```
A B
0 0 -> color 0
0 1 -> color 1
1 0 -> color 2
1 1 -> color 3
```

Each of the 1024 palette entries contains four RGB colors.

This mode is intended for games, demos, photographic conversion, the parrot demo, etc.

The two plane memories still remain physically separate. Only their interpretation at the final color-selection stage changes.

Prefer one unified physical palette representation if this simplifies the implementation, e.g. 1024×4 RGB565 entries, with overlay mode using the appropriate entries. Document the memory tradeoff.

1024×4×RGB565 = 8192 bytes.

Do not hide this memory cost; keep HDL/resource implications documented.

HDL-oriented reference renderer

Keep the raster logic conceptually trivial.

For physical x=0..5 and y=0..9:

```
semiX = (x >= 3)       // 0 or 1
semiY = y >> 1         // 0..4
bitIndex = semiY*2 + semiX
```

Then obtain one bit from each 10-bit mask.

Avoid general division/modulus in the conceptual HDL path.

Document the pixel pipeline approximately as:

```
desktop pixel
     |
generate mask A/B according to current MM mode
     |
SEMI overlay/indexed compositor
     |
palette/color lookup
     |
optional future brightness/alpha processing
     |
RGB output
```

The native 7xtgv specification should describe behavior, not require framebuffer storage.

A hardware implementation may stream the full-resolution desktop pixel into the compositor.

First implementation strategy

Do this incrementally.

1. Write/update a concise native 7xtgv specification document first.
2. Add a small platform-independent reference cell renderer for SEMI.
3. Add unit tests for bit encoding/decoding and raster expansion.
4. Add a desktop image-conversion utility.

5. Generate one static demo asset.
6. Add the smallest possible on-device demo to display it.
7. Only after the image is visibly correct consider integrating SEMI into other 7xtgv demos/runtime code.

Do not refactor the working VT100 terminal as part of this task.

Required tests for encoding

Tests must confirm at minimum:

- ATTR `11CCAABB` extraction
- reconstruction of all 10 bits of mask A
- reconstruction of all 10 bits of mask B
- reconstruction of all 10 bits of palette index
- bit 0 maps to top-left 3×2 semipixel
- bit 1 maps to top-right
- bits 8/9 map to bottom-left/bottom-right
- every SEMI logical pixel expands to exactly 3×2 physical pixels
- all 1024 mask values can be represented
- all 1024 palette indices can be represented
- overlay priority A > B > desktop
- indexed4 mapping is deterministic and documented

Image converter

Create a desktop-side converter, preferably Python for experimentation.

Input:

- PNG/JPEG/etc.
- target currently fixed at 80×24 cells / 160×120 logical SEMI pixels
- optional crop / contain mode

First resample source image to 160×120 using a good area/Lanczos-like resampling method.

For SEMI_INDEXED4 each 2×5 cell contains 10 pixels and may select one of 1024 four-color palettes.

The converter SHOULD CREATE THE 1024 PALETTES FROM THE ACTUAL IMAGE, rather than use a fixed default palette.

Palette optimisation

There are 1920 cells but only 1024 palette entries.

Suggested first algorithm:

1. Partition the 160×120 image into 1920 groups of 2×5 pixels.
2. For every cell, compute an ideal local palette of at most four colors for its ten pixels.
3. Deduplicate identical/near-identical local palettes.
4. If <=1024 unique palettes remain, use them directly.
5. If >1024 remain, cluster/merge the local palettes into a codebook of at most 1024 four-color palettes.
6. Assign each cell the palette giving minimum color error.
7. Quantize its ten pixels to color indexes 0..3.
8. Split those two-bit indexes into the A and B bitplanes.
9. Encode: CHAR_A = maskA & 0xff CHAR_B = maskB & 0xff AA = maskA >> 8 BB = maskB >> 8 CC = paletteIndex >> 8 EXTA = paletteIndex & 0xff ATTR = 0xc0 | (CC<<4) | (AA<<2) | BB

Color-distance calculations should preferably happen in a perceptually better space than raw RGB if a suitable small dependency is available. RGB is acceptable for a first working reference implementation.

Because each cell contains only ten source pixels, do not overengineer the local four-color optimization.

For the global 1024-palette codebook, start with a deterministic and understandable algorithm. Do not launch unbounded k-means/random restart experiments. If multiple strategies seem worthwhile, implement one straightforward baseline, report its image error, and ask the human before spending time on alternatives.

Iterative improvement option

A useful later refinement is Lloyd/codebook iteration:

- assign each cell to its best existing four-color palette
- recompute/improve each palette from all pixels of cells assigned to it
- repeat only a small explicit number of iterations

Do not automatically run large trial/error searches.

Dithering

Do not make dithering mandatory for v1.

First produce an undithered deterministic conversion.

After that, optionally evaluate one simple ordered/error-diffusion method, but preserve the 2×5 cell/palette boundaries and compare visually with the undithered result before keeping it.

Converter outputs

Emit:

1. a preview PNG showing exactly the reconstructed 160×120 SEMI result, optionally also scaled to 480×240 with nearest-neighbour expansion;
2. a palette table;
3. CHAR_A array;
4. CHAR_B array;
5. ATTR array;
6. EXTA array;
7. optionally one compact packed `.7xg`/similar binary asset if useful;
8. a C/C++ header for the embedded demo if that is currently the easiest path.

The generated asset should contain enough metadata/versioning that later format changes do not silently render incorrectly.

Suggested metadata:

- magic
- version
- columns
- rows
- SEMI compositor type
- palette entry count / format

Keep it tiny.

Parrot demo

Use a user-supplied or repository-local colorful parrot/test image if available. Do not spend time searching for assets if one is not present; ask the human to provide the intended source image.

Generate an image-specific 1024-entry palette set.

Display it using SEMI_INDEXED4.

The goal is to show what the cell architecture itself can do:

```
80×24 cells
  ->
160×120 independent SEMI pixels
  ->
each cell has a selected 4-color local palette
  ->
palette selected from an image-optimized 1024-entry codebook
```

```
  ->
physical output 480×240 via exact 3×2 expansion
```

Report:

- number of distinct palette entries actually used
- uncompressed cell-data bytes
- palette bytes
- total demo asset bytes
- reconstruction error metric
- optionally PSNR, but visual output matters more

Expected raw payload with a complete RGB565 1024×4 palette:

```
cell RAM: 1920 × 4 = 7680 bytes
palette: 1024 × 4 × 2 = 8192 bytes
total: 15872 bytes plus tiny header
```

Unused palette entries need not necessarily be serialized in a future packed format, but keep the first implementation simple.

Hard desktop and future smart-terminal windowing

Document but DO NOT fully implement this during the SEMI task.

The conceptual display stack is:

```
full-resolution hard desktop/background
    ->
cell plane B
    ->
cell plane A
    ->
display
```

TEXT/WIND/INST can use independent foreground/background brightness through EXTA.

A later smart-terminal layer may maintain multiple overlapping cell-based windows, clipping and flattening them into the visible cell RAM.

Window state, z-order, clipping and focus belong in smart-terminal software, not in the low-level 7xtgv video hardware.

Inactive windows may initially be visually dimmed simply by changing EXTA brightness, without touching their semantic ink/paper colors.

A later optional alpha/fade-to-desktop effect can be considered separately. Do not conflate brightness with alpha.

Important compatibility constraint

The current VT100 terminal has diverged from the original dual-plane native 7xtgv renderer and is working.

Treat VT100 as a client/front-end and native 7xtgv as a separate display model.

Do not force ANSI underline/reverse/blink/DEC graphics flags into the native `MMXXXXXX` cell format.

Do not break current RunCPM benchmarks/history-bank behavior while developing SEMI.

Human checkpoints

Stop and report to the human at these points:

A. after the native SEMI spec/data layout is written; B. after the software reference renderer and tests pass; C. after the first converter reconstruction preview exists; D. after the first on-device SEMI image appears.

At each checkpoint report what was changed, any assumptions, memory usage, and any design question discovered.

Do not automatically start repeated quality/performance tuning cycles after a checkpoint.